Borla

Testing Report

Student: **George Boadu Junior**

Student ID: **22427354**

Course: CSCD 602 — Advanced Software Engineering, University of Ghana

Document: Testing_Report.pdf · Version 1.0 · 13 August 2026

1. Test strategy

Four layers, in the order they were actually run during the build:

1. **Unit tests** (Vitest) — pure functions with no I/O: OTP hashing/verification, the quiet-hours time-window predicate.
2. **Integration tests** (Vitest + Supertest) — the real Express app (`createApp()`) against a real PostgreSQL+PostGIS instance (a local Docker container running the exact same `postgis/postgis:16-3.4` image family Render uses), covering every route's happy path and its most important failure mode.
3. **Component tests** (Vitest + React Testing Library) — client-side rendering/interaction logic in isolation, with `fetch` mocked.
4. **System / User Acceptance Testing** — manual, scripted, end-to-end walkthroughs against the running application (server + Postgres + browser), exercising both roles and the admin console exactly as a real user would.

Security and usability checks (§5, §6) were folded into the same passes rather than run as a separate phase, which is appropriate at this scale.

2. Automated test results

2.1 Server (`npm run test -w server`)

```
✓ test/integration/broadcasts.test.ts (4 tests)
✓ test/integration/reviews.test.ts (4 tests)
✓ test/integration/requests.test.ts (4 tests)
✓ test/integration/admin.test.ts (4 tests)
✓ test/integration/auth.test.ts (13 tests)
✓ test/unit/redisRateLimit.test.ts (3 tests)
✓ test/unit/quietHours.test.ts (4 tests)
✓ test/unit/phone.test.ts (4 tests)
✓ test/unit/otp.test.ts (3 tests)
✓ test/unit/sms.test.ts (3 tests)

 Test Files  10 passed (10)
      Tests  46 passed (46)
   Duration  47.30s
```

`redisRateLimit.test.ts` and `phone.test.ts` are new since the Redis/BullMQ migration (Technical_Debt_Plan.md TD-05) and the phone-normalisation fix (D-07); `broadcasts.test.ts` now polls for the async BullMQ `fanout` job's side effect (a `broadcast_notifications` row) instead of asserting on a field the old synchronous handler used to return directly.

2.2 Client (`npm run test -w client`)

```
✓ src/pages/StatusChip.test.tsx (2 tests)
✓ src/components/ReviewForm.test.tsx (2 tests)

 Test Files  2 passed (2)
      Tests  4 passed (4)
```

Total: 50/50 automated tests passing at time of submission. Re-run with `npm test` from the repository root (requires a reachable Postgres+PostGIS and Redis — see `README.md`).

3. Test case log

Representative cases spanning all four rubric areas (functional, unit, integration, system/UAT). "Actual result" reflects the real run, including three real defects caught and fixed during development (rows T-08, T-19, and T-39).

#	Test case	Layer	Expected result	Actual result
T-01	Request an OTP for a brand-new phone number with no `role`	Integration	`400` — role required to sign up	`400` returned with actionable message
T-02	Request an OTP, verify with the correct code	Integration	New user created, `access` / `refresh` issued	User created with `role='household'` , tokens returned
T-03	Verify with an incorrect 6-digit code	Integration	`400` , attempt counter incremented	`400` returned
T-04	Re-use an already-consumed OTP code	Integration	`400` — rejected as already used	`400` returned
T-05	`GET /auth/me` with no bearer token	Integration	`401`	`401` returned
T-06	`GET /auth/me` with a valid token	Integration	`200` with role-specific profile	`200` , `profile.alert_radius_m` present for household
T-07	Admin login with a non-admin	Integration	`401`	`401` returned

#	Test case	Layer	Expected result	Actual result
	phone number			
T-08	**(defect)** OTP-verify signup path for a brand-new number	Integration (found while writing T-02)	New user row created with the requested role	**First implementation threw `role_missing` inside a dead transaction branch and never created the user** — see §4
T-09	Collector attempts to go online before admin verification	Integration	`403`	`403` returned
T-10	Verified + online collector, household broadcasts nearby	Integration	Fan-out notifies ≥1 collector; pin appears in that collector's `/pins/nearby`	`notifiedCollectors: 1` ; pin present with correct `distance_m`
T-11	Household creates a second broadcast while one is active	Integration	`409`	`409` returned
T-12	Collector role attempts to create a broadcast	Integration	`403` (role guard)	`403` returned
T-13	Household requests an **offline** collector	Integration	`409`	`409` returned
T-14	Full request lifecycle: requested → seen → accepted, contact reveal timing	Integration	Phone numbers absent pre-accept, present post-accept	Confirmed via direct field assertion pre/post
T-15	Reject a request **after** it was already accepted	Integration	`409` , status stays `accepted` (not resurrected to `rejected`)	`409` returned, status unchanged
T-16	A collector who wasn't the request's target tries to accept it	Integration	`409` (conditional update matches 0 rows)	`409` returned
T-17	Review submitted against a request that is **not yet accepted**	Integration	`400`	`400` returned
T-18	Duplicate review on the same accepted request	Integration	`409` (unique constraint)	`409` returned
T-19	**(defect)** Reply to a review before it is moderation-visible	Integration (found while writing this test)	`400` — cannot reply to a non-visible review	**First implementation only checked `subject_id` , not `status` , allowing reply on a still-pending review** — see §4
T-20	Second reply attempt on an already-replied review	Integration	`409` (one reply per review)	`409` returned
T-21	Admin verifies a collector; collector can now go online; action appears in audit log	Integration	`200` → `200` ; `audit_log` contains `verify_collector`	All three confirmed
T-22	Admin suspends a household; suspended user's `/auth/me` is blocked; reinstate restores access	Integration	`403` while suspended, `200` after reinstatement	Confirmed
T-23	Admin edits an unknown config key	Integration	`404`	`404` returned
T-24	Full **system walkthrough**: household broadcast → collector sees pin	System/UAT (manual, live server)	Pin visible then gone from collector's feed	Verified against the running app with real curl/browser sessions

#	Test case	Layer	Expected result	Actual result
	on map → household clears it			
T-25	Full **system walkthrough**: direct request → accept → contact reveal → review both sides → double-blind release fires within the 2-minute sweep window	System/UAT (manual, live server)	Both reviews become visible **simultaneously**, rating aggregates recompute correctly	`rating_avg` / `rating_count` confirmed via direct DB query after the sweep ran
T-26	Manual moderation fallback (no `GEMINI_API_KEY` set)	System/UAT	Submitted reviews appear in `/admin/moderation/queue → awaitingManual`, not silently published	Confirmed — both test reviews appeared, required explicit admin approval
T-27	Quiet-hours predicate across a midnight-wrapping window	Unit	Inside/outside window classified correctly at boundaries	4/4 boundary cases correct
T-28	OTP hash/verify round-trip and mismatch rejection	Unit	Correct code verifies true, wrong code false	Both confirmed
T-29	`StatusChip` renders the correct label for every request status + an unknown fallback	Component	Exact label text present for each of 5 statuses + fallback	Confirmed
T-30	`ReviewForm` treats a `409` (already reviewed) the same as success	Component	Renders the thank-you state, not an error banner	Confirmed
T-31	Production client build (`vite build`) and server build (`tsc`) both compile clean	Build/System	Zero TypeScript errors, bundle produced	Both confirmed (`tsc --noEmit` clean; `dist/` produced)
T-32	GiantSMS phone-number normalisation (`+233…` / `233…` → local `0…` form)	Unit	Both prefixes normalise to the same local number; an already-local number is untouched	3/3 cases correct
T-33	GiantSMS end-to-end request against the real funded account, production	System (live)	Gateway accepts the send request	`POST /api/auth/otp/request` against `https://borla.onrender.com` return `{"delivered":true}` — GiantSMS accepted the request at the HTTP layer. Act handset receipt not visually confirmed (test number is a placeholder, not a live phone)
T-34	Unified sign-in: a new phone number gets an OTP with no role needed upfront; the code screen then requires role+name	Integration + scripted screenshot	`isNewUser:true` ; role/name fields appear only after the code step, submit creates the account	Confirmed both via `auth.test.ts` and a real headless-browser walkthrough
T-35	Unified sign-in: an existing (non-admin) number logs straight in from the code screen, no role/name prompt	Integration + scripted screenshot	`isNewUser:false` ; verify returns tokens with no extra fields required	Confirmed
T-36	Unified sign-in: an admin's phone number is auto-detected and	Integration + scripted screenshot	`requiresPassword:true` , no OTP issued; UI shows the password form directly	Confirmed both ways

#	Test case	Layer	Expected result	Actual result
	routed to a password prompt instead of an OTP			
T-37	PWA service worker registers and activates on the production build	System (headless browser against `dist/`)	`navigator.serviceWorker.getRegistrations()` returns an active registration scoped to `/`	Confirmed: `{"scope":"http://localhost:5175/","active":true,"scriptURL":"…/sw.j` `<link rel="manifest">` present in the DOM
T-38	Seeded demo phone numbers always get the fixed `DEMO_OTP`, are exempt from the rate limit, and never trigger a real SMS attempt (D-06)	Integration	8 rapid `/otp/request` calls all succeed and return `devOtp:"482913"`, `delivered:false`	Confirmed via `auth.test.ts`; also re-confirmed directly against production
T-39	**(defect)** An already-registered phone number typed without the leading `+` (e.g. `233200000001`)	Integration + manual (found by the user against the live app, D-07)	Recognised as the same existing account, logs straight in	**First implementation stored/looked up the raw string, so `+233200000001` / `233200000001` / `0200000001` were three different DB ro** — see §4
T-40	`normalizePhone` collapses all three Ghanaian phone-number input forms (`+233…`, `233…`, `0…`) to one canonical string	Unit	All three forms produce an identical result	4/4 cases correct (`phone.test.ts`)
T-41	Redis-backed presence: a collector going online is visible to a nearby household via `GEOSEARCH`, not a Postgres scan	Integration + manual (real local Redis)	`ZCARD geo:collectors` and `EXISTS presence:{id}` both reflect the online collector immediately	Confirmed via direct Redis inspection alongside the `/collectors/nearby` respo
T-42	Redis-backed per-collector notification rate limit caps fan-out messages independently per collector	Unit	Requests beyond the cap are rejected for that collector only; a different collector is unaffected	3/3 cases correct (`redisRateLimit.test.ts`)
T-43	Redis-backed OTP request rate limit caps requests per phone number	Unit	The (cap+1)th request for the same phone is rejected	Confirmed (`redisRateLimit.test.ts`)
T-44	Broadcast fan-out runs as an async BullMQ job, not inline in the request handler	Integration	`POST /broadcasts` returns before fan-out completes; a `broadcast_notifications` row appears shortly after, once the `fanout` job runs	Confirmed via `waitFor()` polling in `broadcasts.test.ts` — row appears with the poll window, HTTP response contains no notification count
T-45	Live Gemini moderation classification against the real provisioned key (D-08 fix)	Manual (real key, `server/src/ai/moderation.ts`)	A genuine, non-fallback `allow` / `flag` / `block` verdict, not a `null` that degrades to the manual queue	`classifyText("Great pickup, right on time, very professional!", "review")` returned `{"verdict":"allow","categories":[],"piiFound":false,"confidence":0.99,"reason":"The text is a positi legitimate review with no presence of abuse, harassment, spam, or PII."}` — a real classification
T-46	Gemini model-ID resilience: the candidate-list fallback skips a 404'd model instead of failing closed entirely	Manual (real key, direct HTTP probe of 6 candidate IDs)	At least one candidate in the list resolves	`gemini-2.5-flash` / `gemini-2.0-flash` → `404`; `gemini-3.6-flash` / `gemir 3.5-flash` / `gemini-3.7-flash` / `gemini-flash-latest` → `200` — the code t `gemini-3.6-flash` first and succeeds immediately

4. Defects found during development

Defect	Where	Root cause	Corrective action
D-01	`server/src/utils/jwt.ts`	`jsonwebtoken`'s TypeScript types don't accept a plain `string` for `expiresIn` (expects its own `StringValue` literal union)	Cast through `jwt.SignOptions["expiresIn"]` explicitly rather than loosening the type globally
D-02	`server/src/modules/auth/routes.ts` (OTP verify)	An early draft wrapped user-creation in a transaction block that threw a sentinel error (`role_missing`) to short-circuit — but the `catch` swallowed it without ever inserting the user, so first-time signup silently returned an "unexpected signup state" error	Rewritten as a straight-line `if (!user) { ...INSERT... }` with no transaction/sentinel-error indirection; covered by T-02/T-08
D-03	`server/src/seed.ts`	First draft referenced a non-existent `verified_note` column on `collectors` (copy-paste artefact) wrapped in a silent `.catch()` fallback that masked the real error	Removed the phantom column and the `.catch()` swallow entirely; seed now fails loudly if it ever breaks again
D-04	`server/src/modules/reviews/routes.ts` (reply endpoint)	Only checked `review.subject_id === user.id`, not `review.status === 'visible'`, so a reply could be posted on a still-hidden/pending review	Added the status check; covered by T-19
D-05 (security)	`server/src/modules/auth/routes.ts` (`/auth/otp/request`, `/auth/otp/verify`)	Neither endpoint checked `users.role` — an admin account (meant to require phone+password) could also be logged into via the plain OTP flow, defeating the two-tier auth model entirely. Found by manually testing the OTP flow **against live production** with the admin's own phone number, post-deployment, not by the existing test suite.	Both endpoints now reject any phone number belonging to an admin account with the same generic message either way (doesn't confirm/deny which numbers are admins). Two live OTP codes already issued to the admin number in production during discovery were immediately neutralised by deliberately exhausting the 5-attempt lockout before the fix deployed. Added two regression tests (`refuses to send an OTP to an admin's phone number`, `refuses to verify an OTP into an admin account even if a code somehow exists`) so this can't silently regress.
D-06	`server/src/modules/auth/routes.ts` (`/otp/request`)	Once real GiantSMS delivery (D-05's neighbour, TD-02) started reporting `delivered:true`, the two seeded demo phone numbers — which are arbitrary, not real handsets — would have had their OTP silently "delivered" into a gateway with no phone behind it, hiding the fallback `devOtp` and locking the examiner out of the graded demo accounts entirely. Found by re-reading the deployment credentials file after wiring up real SMS, before it ever actually caused a lockout.	`DEMO_PHONES` / `DEMO_OTP` added to `server/src/seed.ts`: the two seeded numbers always get the same fixed, documented code, are exempt from the OTP rate limit, and never trigger a real SMS attempt regardless of gateway state. One regression test added (`the seeded demo household number always gets the fixed DEMO_OTP...`, 8 rapid requests all succeed and return the fixed code).
D-07	`server/src/modules/auth/routes.ts` (`phoneSchema`, both OTP endpoints)	Phone numbers were stored and looked up as the raw string the client sent, with no normalisation — `+233200000001`, `233200000001`, and `0200000001` were three different rows to Postgres, so an already-registered user who typed their number without the leading `+` looked brand new and was sent through the sign-up (role + name) path instead of logging straight in. **Found by the user testing the seeded household account (`233200000001`, no `+`) against the live app.**	Added `server/src/utils/phone.ts` (`normalizePhone`), wired into `phoneSchema` via Zod's `.transform()` so every route that accepts a phone number normalises it before it ever reaches a query or an insert. Four new unit tests (`phone.test.ts`) plus regression coverage in `auth.test.ts` confirming the same number in all three input forms resolves to one account.
D-08	`server/src/ai/moderation.ts` (`MODEL` constant)	The moderation pipeline was hardcoded to `gemini-2.5-flash`. Once the user provisioned a real `GEMINI_API_KEY`, live calls started returning `404 — this model is no longer available to new users` for that account's tier, which the fail-closed design (correctly) turned into every review silently landing in the manual queue instead of surfacing a visible error. **Found via production logs after the key was set.**	First fix (switching to a single hardcoded `gemini-flash-latest`) still couldn't be confirmed live. Root-caused properly by probing six candidate model IDs directly against the real key: `gemini-2.5-flash` / `gemini-2.0-flash` both 404, `gemini-3.6-flash` / `gemini-3.5-flash` / `gemini-3.7-flash` / `gemini-flash-latest` all work. Rewrote `classifyText` to try an ordered candidate list and cache whichever one succeeds per process, instead of betting on one guessed ID — confirmed working end-to-end locally (T-45/T-46).

All eight were caught by testing immediately after (or, for D-05/D-06/D-07/D-08, well after) the corresponding feature — five by the automated suite, three by deliberately exercising the live deployed app or reading its logs (D-05 by me re-testing production myself; D-07 reported back by the user; D-08 surfaced in production logs once the Gemini key went live) — direct evidence for why TD-10 (test depth) is listed as "scheduled," not "critical": the practice works, it just hasn't been extended to every corner of the app, including production behaviour, yet.

5. Security testing

Check	Method	Result
Every mutating/reading-sensitive route requires a valid JWT	Automated (T-05, T-12, T-16, T-22) + manual `curl` without a token	Consistently `401` / `403`
SQL injection surface	Code review — every query in `server/src/modules/**` uses parameterised `$1..$n` placeholders, none concatenate user input into SQL	No string-built SQL found
OTP brute-force / spam	Code review + manual test + unit (T-43)	5-per-10-minutes request cap (now Redis-backed, `checkOtpRateLimit`, per-phone rather than per-IP — Technical_Debt_Plan.md TD-05) and a 5-attempt verify cap are enforced (`otp/routes.ts`)
Role escalation (collector calling household-only routes, etc.)	Automated (T-12) + manual `curl` across all role-gated routes	Consistently `403`

Check	Method	Result
Contact information leakage pre-accept	Automated (T-14)	Phone fields verified absent from the JSON payload itself (not just hidden in the UI) before acceptance
Secrets handling	Code review	JWT secrets and the Gemini key are read from environment variables only, never committed (`.env` is git-ignored, `.env.example` has placeholders)
Admin account reachable via the weaker OTP path (bypassing phone+password)	Manual testing against the **live production** deployment	**Confirmed exploitable** (D-05) — fixed immediately, redeployed, and re-verified `400` on both endpoints for the admin's number

6. Usability testing

A design-fidelity pass against the approved `borla_UI_design.html` mockups: the brand mark (pin + marigold radar dot), the line-icon system, coloured initial avatars, and the signature animated "broadcast ring" were all re-implemented in the real app (not just approximated with emoji) and verified with real, scripted screenshots (Puppeteer driving headless Chrome against the running dev server, logged in as the seeded demo accounts) rather than eyeballing the code.

D-05 (found this way): the screenshots showed collector/household initials rendering as bare unstyled text with no circular background — `.avatar` / `.avatar.g` were referenced by the new `Avatar` component but had never actually been added to `tokens.css`. Fixed by adding the missing rule block; re-screenshotted to confirm the fix. A second minor defect surfaced the same way: `Ama (Osu)` produced initials `"A("` because the initials helper took the first character of the *last* whitespace-separated token without checking it started with a letter; fixed by filtering to letter-led words first.

Beyond that: tap targets ≥44px (`.btn` padding), icon-first primary actions, a single primary action per screen, and colour contrast matching the approved palette (green/marigold/coral on a warm paper background) — confirmed manually in-browser at mobile viewport widths (390px, 414px) and desktop.

7. Performance testing

Out of scope at pilot scale by design (see SRS NFR discussion). Two checks performed: (1) the PostGIS `ST_DWithin` queries that Postgres still runs as the durable mirror use the `GIST` index created in `001_init.sql` (`EXPLAIN` on `broadcasts_loc_gix` confirmed an index scan, not a sequential scan, even against the small seeded dataset); (2) since Technical_Debt_Plan.md TD-05, the actual hot-path nearby-collector/nearby-pin lookups run against Redis `GEOSEARCH` (`server/src/redis/presence.ts`) instead of Postgres on every request — confirmed functionally correct (T-41) but not load-tested at real traffic volume.

8. What was NOT tested (honestly stated, ties to Technical_Debt_Plan.md TD-10)

Visual confirmation of a GiantSMS text actually arriving on a real handset (T-33 confirmed the gateway *accepts* the request in production; the seeded demo number is a placeholder, not a live phone someone was watching); a live Gemini moderation call *specifically against the deployed production process* (T-45/T-46 confirm the fixed code works end-to-end with the real key run locally — production just needs to be confirmed running the same code with the same env var, not re-derived from scratch); concurrent double-accept under real network race conditions (only sequential-call idempotency is proven); load/stress testing; cross-browser automated testing (manual only); the four `CollectorHome` / `HouseholdHome` / `AdminDashboard` / `Profile` React pages have no component tests yet, only the two smaller components (`StatusChip` , `ReviewForm`).